# C5a Python with SQLite

Relational Database

# Part 1 : Create Database and Tables

Python with SQLite



# Create database

```
from sqlite3 import *

open('database.db','w').close()

db = connect('database.db')
```

To overwrite an existing file.

* Sqlite do not support DROP DATABASE

Open `database.db` if exist, else create a new one.

```
db.close()
```

Close the database file




# Entity Relation (ER) Diagram




**Table Description :**

Products(Id, ProductName, UnitPrice, Package, SupplierName, SupplierCountry)

Customers (Id, FirstName, LastName, City, Country, Phone)

Orders (Id, OrderDate, CustomerId*, ProductId*, Quantity)

__ : Primary Key;

* : Foreign Key

Execute SQL

# Create Tables using SQL Commands




```
CREATE TABLE Products (
Id                    INTEGER NOT NULL,
ProductName           TEXT,
UnitPrice             NUMERIC,
Package               TEXT,
SupplierName          TEXT,
SupplierCountry       TEXT,
PRIMARY KEY(Id)
);
```

```
CREATE TABLE Customers (
Id              INTEGER NOT NULL,
FirstName       TEXT NOT NULL,
LastName        TEXT NOT NULL,
City            TEXT,
Country         TEXT NOT NULL,
Phone           TEXT NOT NULL,
PRIMARY KEY(Id)
);
```

```
CREATE TABLE Orders (
Id              INTEGER NOT NULL,
OrderDate       TEXT NOT NULL,
CustomerId      INTEGER NOT NULL,
ProductId       INTEGER NOT NULL,
Quantity        INTEGER NOT NULL,
FOREIGN KEY(ProductId) REFERENCES Products(Id),
FOREIGN KEY(CustomerId) REFERENCES Customers(Id),
PRIMARY KEY(Id)
);
```

# Create tables

```
from sqlite3 import *

db = connect('database.db')

c = db.cursor()
c.execute('''CREATE TABLE Products(\
        Id INTEGER NOT NULL, ProductName TEXT,\
        UnitPrice NUMERIC, Package TEXT,\
        SupplierName TEXT, SupplierCountry TEXT,\
        PRIMARY KEY(Id)\
        );''')

db.commit()
db.close()
```

Python will ignore the newline when there is a backslash '\' here.

Save the change

**TryIt!**

Create the `Customers` and `Orders` tables.

`2 create table - Customers & Orders.py`

# Part 2 : Insert Data into Tables

Python with SQLite



Execute SQL

# Insert one row of data into table




```
INSERT INTO Products
      (ProductName, UnitPrice, Package, SupplierName, SupplierCountry)
VALUES
      ('Chai', 18, '10 boxes x 20 bags', 'Exotic Liquids', 'UK');
```

| Id | ProductName | UnitPrice | Package | SupplierName | SupplierCountry |
|---|---|---|---|---|---|
| 1 | Chai | 18 | 10 boxes x 20 bags | Exotic Liquids | UK |

- `Id` field will be populated by auto-increment generated values

# Insert one data into the table using Tuple




Use database.db and ensure that the Products table is empty.

```
from sqlite3 import *
db = connect('database.db')

c = db.cursor()
c.execute('''INSERT INTO Products (ProductName,\
    UnitPrice, Package, SupplierName, SupplierCountry) \
    VALUES(?,?,?,?,?)''',
    ('Chai', 18, '10 boxes x 20 bags', 'Exotic Liquids', 'UK'))

db.commit()
db.close()
```

Note that execute() takes 2 arguments:
a SQL command string and a tuple containing the data.

Execute SQL

# Insert multiple rows of data into table




```
INSERT INTO Products
      (ProductName, UnitPrice, Package, SupplierName, SupplierCountry)
VALUES
      ('Chang', 19, '24 - 12 oz bottles', 'Exotic Liquids', 'UK'),
      ('Aniseed Syrup', 10, '12 - 550ml bottles', 'Exotic Liquids', 'UK'),
      ("Chef Anton's Cajun Seasoning", 22, '48 - 6 oz jars', 'New Orleans
Cajun Delights', 'USA'),
      ("Chef Anton's Gumbo Mix", 21.35, '36 boxes', 'New Orleans Cajun
Delights', 'USA');
```

Use double quote " if data contains single quote '

# Insert multiple records from a data list

```
datalist = [('Chang', 19, '24 - 12 oz bottles', 'Exotic Liquids',
'UK'), ('Aniseed Syrup', 10, '12 - 550ml bottles', 'Exotic
Liquids', 'UK'), ("Chef Anton's Cajun Seasoning", 22, '48 - 6 oz
jars', 'New Orleans Cajun Delights', 'USA'), ("Chef Anton's Gumbo
Mix", 21.35, '36 boxes', 'New Orleans Cajun Delights', 'USA')]
```

```
for data in datalist:
    c.execute('''INSERT INTO Products (ProductName,\
    UnitPrice, Package, SupplierName, SupplierCountry)\
    VALUES (?,?,?,?,?)''', data)
```




Remember to include the 5 necessary lines of code:

```
from sqlite3 import *
db = connect('database.db')
c = db.cursor()
db.commit()
db.close()
```

# Insert data from a csv file

```
from sqlite3 import *
from csv import *
db = connect('database.db')
c = db.cursor()

f = open("Products.csv")
lst = reader(f)
next(lst)    #skip the header in csv

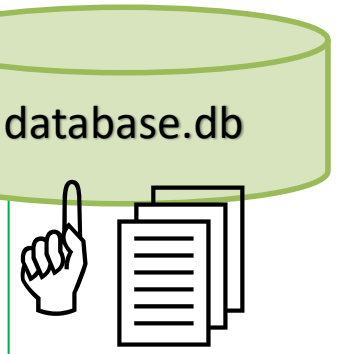
for data in lst:
    c.execute('''INSERT INTO Products (Id, ProductName,\
    UnitPrice, Package, SupplierName, SupplierCountry) \
    VALUES(?,?,?,?,?,?)''', data)
f.close()
db.commit()
db.close()
```

**TryIt!**

Insert all the data for the `Customers` and `Orders` tables using the Customers.csv and Orders.csv files.

`5 insert data from csv - Customers & Orders.py`

The data for Id is included in the csv file.




Important!!!
Ensure that the Products table is empty (delete all the previously inserted records first)

We will usually only update specific record(s) in the table.

# Part 3 : Update Data

Python with SQLite



# Using UPDATE and SET

## Change Thomas Hardy's Country from UK to Singapore.

We will usually only update specific record(s) in the table.

```
c.execute('''UPDATE Customers SET Country=?\
          WHERE FirstName="Thomas" \
          AND LastName="Hardy";''', ('Singapore',))
```

Remember to include the 5 necessary lines of code:

```
from sqlite3 import *
db = connect('database.db')
c = db.cursor()
db.commit()
db.close()
```

This data tuple contains one element to be used for '?' in the SQL command.



# Using UPDATE and SET

Change the name of Customer ID 5 to Harry Potter.

We will usually only update specific record(s) in the table.

```
c.execute('''UPDATE Customers \
          SET FirstName=?, LastName=?\
          WHERE Id=5;''', ('Harry','Potter'))
```

Remember to include the 5 necessary lines of code:

```
from sqlite3 import *
db = connect('database.db')
c = db.cursor()
db.commit()
db.close()
```

This data tuple contains two elements, 'Harry' for the first '?' and 'Potter' for the second '?' in the SQL command.



# Using UPDATE and SET

Increase the quantities for the first 4 orders by 10.

We will usually only update specific record(s) in the table.

```
c.execute('''UPDATE Orders \
          SET Quantity = Quantity + ? \
          WHERE Id<5;''', (10,))
```

This data tuple contains one element to be used for '?' in the SQL command.

Remember to include the 5 necessary lines of code:

```
from sqlite3 import *
db = connect('database.db')
c = db.cursor()
db.commit()
db.close()
```



We will usually only delete specific record(s) in the table.

# Part 4 : Delete Data

Python with SQLite



# Delete Record(s) in a Table

```
c.execute('''DELETE FROM Customers \
          WHERE FirstName = 'Victoria';''')
```

We will usually only delete specific record(s) in the table.

Remember to include the 5 necessary lines of code:

```
from sqlite3 import *
db = connect('dummy.db')
c = db.cursor()
db.commit()
db.close()
```

```
DELETE FROM Customers;
```

**DANGER:** This will delete all the records in the table but the Table will still remain.

```
c.execute('''DELETE FROM Customers;''')
```




```
DROP TABLE Customers;
```

```
c.execute('''DROP TABLE Customers;''')
```

# Part 5 : Query

Python with SQLite



# Using SELECT with fetchone() and fetchall()




```
c.execute('''SELECT * FROM Products WHERE Id=1''')
item = c.fetchone()
print(item)
```

```
c.execute('''SELECT * FROM Customers''')
items = c.fetchall()

for item in items:
    print(item)
```

No need to use commit() since there is no change to the data in the tables.

Remember to include the 5 necessary lines of code:

```
from sqlite3 import *
db = connect('Northwind.db')
c = db.cursor()
db.commit()
db.close()
```

**TryIt!**

Use these query statements with fetchone() and fetchall().

8 fetchone and fetchall – Exercises.py




```
SELECT * FROM Products;
SELECT Id, ProductName, UnitPrice FROM Products;
SELECT DISTINCT SupplierName FROM Products;
SELECT * FROM Customers WHERE Country = 'Mexico';
SELECT ProductName, UnitPrice FROM Products WHERE UnitPrice BETWEEN 10 AND 20;
SELECT FirstName FROM Customers WHERE FirstName LIKE '%ja%';
SELECT * FROM Customers WHERE Country IN ('Germany', 'France', 'UK')
SELECT ProductName, UnitPrice FROM Products ORDER BY UnitPrice DESC LIMIT 10;
SELECT COUNT(ProductName), SupplierCountry FROM Products GROUP BY SupplierCountry;
SELECT SUM(Quantity) FROM Orders;
```

The next 5 slides are for you to recall some of the query statements done in C5a Basic SQL.

Execute SQL

# Using WHERE with other Operators




| Operator | Description | Example |
|---|---|---|
| BETWEEN | Between a certain range | SELECT ProductName, UnitPrice FROM Products<br>WHERE UnitPrice BETWEEN 10 AND 20; |
| LIKE | Search for a pattern | SELECT FirstName FROM Customers<br>WHERE FirstName LIKE '%ja%' |
| IN | Multiple OR conditions | SELECT * FROM Customers<br>WHERE Country IN ('Germany', 'France', 'UK') |

Wildcard Character

Wildcard Characters % and _ :

bl% finds bl, black, blue and blur

h_t finds hot, hat and hit



Execute SQL

# Using ORDER BY and LIMIT




```
SELECT ProductName, UnitPrice FROM Products
ORDER BY UnitPrice DESC LIMIT 10;
```

Top 10 most expensive products

```
SELECT FirstName, Country FROM Customers
ORDER BY Country ASC, FirstName DESC;
```

The ORDER BY sorts the records in ascending order by default.



Execute SQL

# Using GROUP BY




Observe the outputs from the following:

```
SELECT ProductName, SupplierCountry FROM Products;

SELECT ProductName, SupplierCountry FROM Products
GROUP BY SupplierCountry;

SELECT COUNT(ProductName), SupplierCountry FROM Products
GROUP BY SupplierCountry;

SELECT COUNT(ProductName), SupplierCountry FROM Products
GROUP BY SupplierCountry
ORDER BY COUNT(ProductName) DESC;
```






Execute SQL

# Aggregate Functions




- COUNT, AVG, SUM, MAX, MIN

```
SELECT COUNT(ProductID) FROM Products;
SELECT AVG(UnitPrice) FROM Products;
SELECT SUM(Quantity) FROM Orders;
SELECT MAX(UnitPrice) FROM Products;
SELECT MIN(UnitPrice) FROM Products;
```



Execute SQL

# Using HAVING

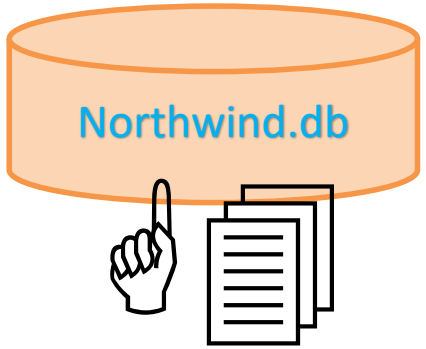



```
SELECT COUNT(Id), Country FROM Customers
GROUP BY Country
HAVING COUNT(Id) > 5
ORDER BY COUNT(Id) DESC;
```

Note: SQL WHERE clause cannot be used with aggregate functions.

OUTPUT:

| COUNT(Id) | Country |
|---|---|
| 13 | USA |
| 11 | France |
| 11 | Germany |
| 9 | Brazil |
| 7 | UK |



# *The End*

C5a Python with SQLite